

Multi-signature in Fully Split Ring and Quantum Random Oracle Model

Abstract

Multi-signature schemes are crucial for secure operations in digital wallets and escrow services within smart contract platforms, particularly in the emerging post-quantum era. Existing post-quantum multi-signature constructions either do not address the stringent requirements of the Quantum Random Oracle Model (QROM) or fail to achieve practical efficiency due to suboptimal parameter choices.

In this paper, we present a novel multi-signature scheme designed to be secure in the QROM and optimized for practical use. Our scheme operates over the polynomial ring $\mathbb{Z}_q[X]/(x^n + 1)$ with $q \equiv 1 \pmod{2n}$, enabling full splitting of the ring and allowing for efficient polynomial arithmetic via the Number Theoretic Transform (NTT). This structure not only ensures post-quantum security but also bridges the gap between theoretical constructs and real-world implementation needs.

We further propose a new hardness assumption, which is termed ν -SelfTargetMSIS, extending SelfTargetMSIS (Eurocrypt 2018) to accommodate multiple challenge targets. We prove its security in the QROM and leverage it to construct a secure and efficient multi-signature scheme. Our approach avoids the limitations of previous techniques, reduces security loss in the reduction, and results in a more compact and practical scheme suitable for deployment in post-quantum cryptographic systems.

Keywords

Multi-signature, NTT, Quantum Random Oracle Model

ACM Reference Format:

. 2025. Multi-signature in Fully Split Ring and Quantum Random Oracle Model. In . ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

With the development of science and technology, it becomes more and more realistic that quantum computers will come to our life sooner or later. Since the traditional cryptographic system will be broken when quantum computers come, the US National Institute of Standards and Technology (NIST) announced three selected digital signatures in the post-quantum signature standardization on July 5, 2022: CRYSTALS-Dilithium, FALCON, and SPHINCS+. On August 13, 2024, NIST published the Federal Information Processing Standard (FIPS) 204 for the CRYSTALS-Dilithium algorithm [21],

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

which has been renamed as Module-Lattice-Based Digital Signature Algorithm (ML-DSA). CRYSTALS-Dilithium is the one with more studies in academia because of its good structure and the worst-case to average-case reduction.

Quantum computers have not only the ability to attack traditional cryptosystems, but also the power to query a function with multiple inputs at a *single query*. Currently, most signature schemes rely on a hash function to turn message(s) into challenge(s), and thus sign them. However, many of them use the random oracle model (ROM) to simulate the output of a hash function in their security proofs (e.g., [15]). When we consider the case of a quantum adversary, it always makes a query in superposition, which makes it hard for the simulator to find out what preimage the adversary is interested in. More threateningly, the adversary could make an attack on superposition that comprises different messages in one move. This uncertainty of such powerful adversaries calls eagerly for the security reductions over the quantum random oracle model (QROM) [4].

QROM-based Multisignatures. In this paper, we consider the case of *multi-signature* - an aggregation of signatures to show that multiple users agree on the same message. It is widely used in cryptocurrency and blockchain technologies. For the case of a digital wallet for cryptocurrency, a wallet could require multi-signatures (from any two public keys out to three) to authorize spending. So even if one secret key is stolen, the attacker cannot access the funds. It is also useful for escrow services in a smart contract. With a 2-out-of-3 multi-signature with a buyer, seller, and a neutral arbitrator, funds are only released when two of the three agree, avoiding the need to trust a single party.

Currently, there already exist some multi-signature solutions based on lattice-based hard problems [5, 7, 9, 12, 16], but few have the support for the quantum random oracle model [12, 16]. Such a need brings us to QROM-based multi-signatures.

Towards practical Multisignatures. Dilithium [11] is the most famous post-quantum secure signature scheme constructed via the "Fiat-Shamir with aborts" framework [18]. It is computed in a polynomial ring $\mathcal{R}_q := \mathbb{Z}_q[X]/(X^n + 1)$, where q is a prime modulus. A QROM-based Dilithium was proposed in [15]. However, this QROM-based Dilithium faces a critical threat as pointed out by [6].

For the current QROM-based multi-signature relying on Dilithium [12], the chosen parameters are good for invertibility and make the security proof easy. More specifically, it sets the prime number by $q \equiv 5 \pmod{8}$. The choice of this parameter started from [20], with the underlying polynomial ring being $\mathbb{Z}_q[X]/(X^n + 1)$ and n being a power of two. Moreover, the invertibility comes from the isomorphism

$$\mathbb{Z}_q[X]/(X^n + 1) \cong \mathbb{Z}_q[X]/(X^{n/2} - r) \times \mathbb{Z}_q[X]/(X^{n/2} + r)$$

where the $\mathbb{Z}_q[X]/(X^{n/2} \pm r)$ are fields because both of them are unsplitable. However, this structure is not good for multiplication implementation. A naive polynomial multiplication is $O(n^2)$. It

is hard to take Number-Theoretic Transform (NTT) acceleration similar to the underlying Dilithium, which is only $O(n \log n)$.

Practical Dilithium would always choose a faster ring with $q \equiv 1 \pmod{2n}$ as in [3]. The ring is fully split as $\mathbb{Z}_q[X]/(X^n + 1) \cong \mathbb{Z}_q^{n \times n}$, and NTT could make good use of this isomorphism [3]. The hardness of this scheme is based on the hardness of three computational problems: Module Learning with Errors (MLWE), Module Short Integer Solution (MSIS), and SelfTargetMSIS. MLWE and MSIS are widely believed to be secure. In 2024, Jackson et al. [14] presented a reduction of the hardness of SelfTargetMSIS to MLWE in the QROM model. In this paper, we extend the assumption of SelfTargetMSIS, and give the security proof of a multi-signature with compact parameters.

1.1 Related works

Researches on QROM. There have already been some researches on QROM in the past decade. Following up with [23], lots of researchers agree that it is almost impossible to rewind the quantum machine, compared to the classical random oracle model, and it is inefficient to prove 2-soundness and similar properties.

After Zhandry [24] remarked that QROM could be easily simulated by $2q$ -wise independent functions, there are a lot of trials presenting more reliable properties of QROM and building schemes based on those properties. For example, it is shown in [22] how to adaptively reprogram a hash function. But still, the simulation, extraction, and rewinding are considered to be hard because of the superposition of quantum queries [17].

In 2019, Zhandry [25] proposed the compressed oracle model. This model uses lazy sampling and supposes that the hash outcome $H(x)$ of any input x is a uniform superposition like $\sum |y\rangle$ before taking measurements. If anybody wants to know the result, it must measure this register and further impact the register of a simulator. It gives the simulator more power to control and guess the inputs and outputs of hash queries. In [8], the authors provided an efficient simulation of the compressed oracle.

QROM-Dilithium and NTT. The QROM-Dilithium was raised in [15], in which the security could rely on either lossy key generation or SelfTargetMSIS. The lossy key generation relies firmly on $q \equiv 5 \pmod{8}$, while the SelfTargetMSIS reduction could cater to different needs. However, it only showed that SelfTargetMSIS is secure in the random oracle model in [15]. In 2024, [14] showed a hardness proof on SelfTargetMSIS in the QROM model. Using [10], the authors showed SelfTargetMSIS is at least as hard as MLWE in the given parameter. Furthermore, it demonstrated that using $q \equiv 1 \pmod{2n}$ with NTT acceleration would accelerate the original QROM-Dilithium several times. This QROM-based Dilithium faces a critical threat as pointed out by [6].

Other QROM Multisignature schemes. To the best of the authors' knowledge, there are two researches on QROM multi-signatures. The first one [12] provides a simple extension of the Dilithium to multi-signature. It uses [22] as the main reprogramming theory. After reprogramming, the authors proved that the schemes' unforgeability under chosen message attack (MS-UF-CMA) is almost equivalent to the unforgeability under no message attack (MS-UF-NMA). And to prove the MS-UF-NMA of the signature scheme,

the authors used the MLWE assumption to make the keys lossy. Then the authors presented a general search problem with bounded model such that no quantum adversary could easily attack the lossy MS-UF-NMA of the scheme.

The second QROM multi-signatures [16] provides a two-round solution on multi-signature, which relies on [22] for online-extractability. But it still uses a ring that does not provide fast NTT acceleration.

1.2 Our Contributions

There are two main contributions in our paper. To illustrate our contribution, we define some notations first. Apart from the ring $\mathcal{R}_q$ with $q \equiv 1 \pmod{2n}$ and n is a power of two, we also define $\mathcal{R} := \mathbb{Z}[X]/(X^n + 1)$. For any $a \in \mathbb{Z}_q$, $a' \pmod{\pm q}$ is defined by the unique $-(q-1)/2 \leq a' \leq (q-1)/2$ such that $a' \equiv a \pmod{q}$. For any $r = r_0 + r_1X + \dots + r_{n-1}X^{n-1} \in \mathcal{R}_q$, we use the notation $\|r\|_i := |r_i \pmod{\pm q}|$ and $\|r\|_\infty := \max_i \|r_i\|_i$. For $\mathbf{r} = (r_1, \dots, r_m) \in \mathcal{R}_q^m$, we have $\|\mathbf{r}\|_\infty := \max_i \|r_i\|_\infty$. For $\tau \in \mathbb{N}$, B_τ denotes the set of all elements in $r \in \mathcal{R}_q$ with $\|r\|_\infty = 1$ and the number of ± 1 coefficients is indeed τ . $\mathbf{I}_m$ is an identity matrix of size m .

Contribution 1. We propose a variant to the SelfTargetMSIS, named v -SelfTargetMSIS, and present a hardness proof for this variant. In the original SelfTargetMSIS, the adversary is given a matrix $\mathbf{A} \in \mathcal{R}_q^{m \times k}$ with a hash function $H : \{0, 1\}^* \rightarrow B_\tau$, it needs to find out a message M and corresponding $\mathbf{y} \in \mathcal{R}^{m+k}$ such that $\|\mathbf{y}\|_\infty$ is small, $H([\mathbf{I}_m \quad \mathbf{A}] \mathbf{y}, M) = y_{m+k}$, of which y_{m+k} is the $(m+k)$ -th element of $\mathbf{y}$.

However, single hash target y_{m+k} is not suitable for multi-signature. In our variant v -SelfTargetMSIS, the adversary needs to find out a solution for v different targets at the same time. Precisely, it is given $\mathbf{A} \in \mathcal{R}_q^{m \times k}$ and hash function $H : \{0, 1\}^* \rightarrow B_\tau$. The adversary needs to find out a solution $\mathbf{A}' \in \mathcal{R}_q^{m \times (v-1)}$, M , and $\mathbf{y} \in \mathcal{R}^{m+k+v-1}$ such that $\|\mathbf{y}\|_\infty$ is small, and

$$\forall i \in [v], H(i, \mathbf{A}', [\mathbf{I}_m \quad \mathbf{A} \quad \mathbf{A}'] \mathbf{y}, M) = y_{(m+k-1)+i}.$$

Contribution 2. We propose a multi-signature scheme based on [12]. Their scheme relies on the security of $q \equiv 5 \pmod{8}$ at Lemma 4.6 of [15], and further on Lemma 2.2 of [19]. Although the lemma provides invertibility, it is insufficient for NTT acceleration. We prove that our multi-signature scheme is secure at $q \equiv 1 \pmod{2n}$, by using v -SelfTargetMSIS. And instead of using only high bits of $\mathbf{A}\mathbf{y}$, we use $\mathbf{A}\mathbf{y}_1 + \mathbf{y}_2$ to work around the threat posed by [6].

Moreover, [12] uses a lemma for getting reprogrammable from [22]. The lemma in [22] is stateless and simulated by a polynomial. It further implies the underlying hash function H_2 is length-preserving. And it also needs an additional almost compressed oracle to help [17], which brings more losses. To get rid of these limitations, we replace the [22] reprogramming with [13]. This change also reduces MS-UF-NMA to MS-UF-CMA of the multi-signature and gets rid of the almost compressed oracle, which gives a better loss.

2 Preliminaries

2.0.1 Notations. For $\eta \in \mathbb{N}$, the notation $S_\eta \in \mathcal{R}_q$ is the set of all polynomials such that the coefficients all belong to $\{-\eta, \dots, \eta\}$. Besides, we use $\text{Adv}_{\text{params}}^{\text{SCHEME}}(\mathcal{A}) := \Pr[x \text{ passes the verification} | x \leftarrow \mathcal{A}]$ to denote the advantage that an adversary $\mathcal{A}$ could break a search version hard problem of the scheme SCHEME on forging. We use $\text{negl}(\lambda)$ to represent a negligible function in λ .

2.1 Quantum random oracle model (QROM)

A state of a qubit $|\phi\rangle$ could be expressed as a two-dimensional complex vector $|\phi\rangle = \alpha|0\rangle + \beta|1\rangle$ where $\{|0\rangle, |1\rangle\}$ is an orthogonal basis of $\mathbb{C}^2$ and $|\alpha|^2 + |\beta|^2 = 1$. The qubit is in superposition when $0 < |\alpha| < 1$. Moreover, the state of n qubits could be written as $|\phi\rangle = \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle$ in $\mathbb{C}^{2^n}$ and $\sum_x |\alpha_x|^2 = 1$. When measuring the state in the computational basis, it outputs x with probability $|\alpha_x|^2$.

One of the most important observations is that it is able to replace any hash function by a fixed polynomial following the Lemma 2.1.

LEMMA 2.1 (LEMMA IN [24]). *Let $H : X \rightarrow \mathcal{Y}$. There is no quantum polynomial time (QPT) adversary $\mathcal{A}^H$ that could distinguish a real hash $|H\rangle$ and $2Q$ -wise independent function f_{2Q} when making at most Q quantum queries to the random oracle. And the f_{2Q} could be treated as a $2Q$ degree polynomial in $\mathbb{F}_Y$.*

2.2 Lattice

In the original QROM-Dilithium [15], the security level of the Dilithium protocol is given by the advantages of module lattice $\text{Adv}_{k,l,\eta}^{\text{MLWE}}(\mathcal{B})$, $\text{Adv}_{H,\tau,k,t+1,\zeta}^{\text{SelfTargetMSIS}}(\mathcal{C})$, and $\text{Adv}_{k,l,\zeta'}^{\text{MSIS}}(\mathcal{D})$. Thus, we rephrase the definition of these hardness assumptions in the style of [14].

Definition 2.2 (Module Learning with Errors (MLWE)). For $m, k, \eta \in \mathbb{N}$, the advantage that any adversary $\mathcal{A}$ breaking the hard assumption is

$$\text{Adv}_{m,k,\eta}^{\text{MLWE}}(\mathcal{A}) := \left| \Pr \left[b = 0 \mid \mathbf{A} \leftarrow \mathcal{R}_q^{m \times k}, t \leftarrow \mathcal{R}_q^m, b \leftarrow \mathcal{A}(\mathbf{A}, t) \right] - \Pr \left[b = 0 \mid \mathbf{A} \leftarrow \mathcal{R}_q^{m \times k}, (s_1, s_2) \leftarrow S_\eta^k \times S_\eta^m, t \leftarrow \mathbf{A}s_1 + s_2, b \leftarrow \mathcal{A}(\mathbf{A}, t) \right] \right|.$$

Definition 2.3 (Module Short Integer Solutions (MSIS)). For $m, k, \gamma \in \mathbb{N}$, the advantage that any adversary $\mathcal{A}$ breaking the MSIS is defined as

$$\text{Adv}_{m,k,\gamma}^{\text{MSIS}}(\mathcal{A}) := \Pr \left[0 < \|\mathbf{y}\|_\infty \leq \gamma \wedge [\mathbf{I}_m | \mathbf{A}]\mathbf{y} = \mathbf{0} \mid \mathbf{A} \leftarrow \mathcal{R}_q^{m \times k}; \mathbf{y} \leftarrow \mathcal{A}(\mathbf{A}) \right].$$

Definition 2.4 (SelfTargetMSIS). For $\tau, m, k, \gamma \in \mathbb{N}$ and $H : \{0, 1\}^* \rightarrow B_\tau \subset \mathcal{R}_q$, the advantage that any adversary $\mathcal{A}$ breaking the Self-TargetMSIS is defined as

$$\text{Adv}_{H,\tau,m,k,\gamma}^{\text{SelfTargetMSIS}}(\mathcal{A}) := \Pr \left[\|\mathbf{y}\|_\infty \leq \gamma \wedge H([\mathbf{I}_m | \mathbf{A}]\mathbf{y}, M) = c \mid \mathbf{A} \leftarrow \mathcal{R}_q^{m \times k}; \left(\mathbf{y} := \begin{bmatrix} \mathbf{r} \\ c \end{bmatrix}, M \right) \leftarrow \mathcal{A}^{[H]}(\mathbf{A}) \right].$$

The adversary has quantum access to the H hash function.

2.3 Multi-signature

In multi-signature with the maximum signer size τ , every individual participant could generate their own key pair, participate in *interactively* signing, and make verifications. We use the notation similar to [5], without using key aggregation and online/offline signing.

- $\text{pp} \leftarrow \text{Setup}(1^\lambda)$ takes a security parameter 1^λ as input, and outputs public parameters. Throughout, we assume that pp is given as implicit input to all other algorithms.
- $(\text{sk}, \text{pk}) \leftarrow \text{KeyGen}()$ generates a secret key sk and a public key pk .
- $\sigma \leftarrow \text{Sign}(\text{sk}, \text{pk} := \{\text{pk}\}, m)$ signs interactively on message m with signers who own the secret keys of pk where $|\text{pk}| \leq \tau$, and outputs a signature σ .
- $\{0, 1\} \leftarrow \text{Verify}(\text{pk}, \sigma, m)$ verifies whether the signature σ is sign by the secret owners of all pk on the message m and $|\text{pk}| \leq \tau$.

Definition 2.5. A multi-signature scheme Scheme is MS-UF-CMA safe if the advantage any adversary $\mathcal{A}$ wins the given experiment with negligible probability.

$$\text{Adv}_{\text{Scheme}}^{\text{MS-UF-CMA}} := \Pr \left[1 \leftarrow \text{Exp}_{\text{Scheme}}^{\text{MS-UF-CMA}}(\mathcal{A}, \lambda) \right] \leq \text{negl}(\lambda),$$

where the experiment $\text{Exp}_{\text{Scheme}}^{\text{MS-UF-CMA}}(\mathcal{A}, \lambda)$ is defined as

- (1) $\text{pp} \leftarrow \text{Setup}(1^\lambda), (\text{sk}^*, \text{pk}^*) \leftarrow \text{KeyGen}()$.
- (2) $(\text{pk}^*, \sigma^*, m^*) \leftarrow \mathcal{A}^{O_{\text{Sign}}}(\text{pp}, \text{pk}^*)$.
- (3) Return $\text{Verify}(\text{pk}^*, \sigma^*, m^*) = 1 \wedge \text{pk}^* \in \text{pk} \wedge (\text{pk}^*, m^*) \notin Q$.

The oracle $O_{\text{Sign}}(\text{pk}, m)$ aborts if $\text{pk}^* \notin \text{pk}$. Otherwise, it sets the queried set $Q \leftarrow Q \cup (\text{pk}, m)$ and answers all interactive queries by $\text{Sign}(\text{sk}^*, \text{pk}, m)$.

Definition 2.6. A multi-signature scheme's MS-UF-NMA is the same as MS-UF-CMA except that no O_{Sign} query could be made.

2.4 Compressed oracles and reprogramming

The compressed oracle is built to record all the information added by the adversary. Unlike the classical method, the update and response procedures are all considered in the superposition. More precisely, if the adversary makes $\sum |x\rangle$ to a random oracle, the simulator could make it entangled with its own database D by answering $\sum_{x,u,D} |x, u\rangle \times |D\rangle$ with $\sum_{x,u,D} |x, u \oplus D(x)\rangle \times |D\rangle$. When the adversary tries to obtain the hash of a single value, it needs to measure $|x\rangle$ register and consequently fix $D(x)$ in the simulator.

The reprogrammability inside quantum random oracles is studied in [2]. Although this paper gets a better bound on the loss, we require an adaptive variant such that the reprogrammed domain could have been impacted by the adversary. The variant we use in our paper is as follows [13], and it is built on top of a compressed oracle. Here, the probability difference is the advantage that any distinguisher knows the oracle has been reprogrammed.

LEMMA 2.7 (PROPOSITION 1 IN [13]). *Let X_1, X_2, Y be finite sets, $\mathcal{D}$ be any distinguisher making at most q queries to H , and R be the number of reprogramming points. The underlying hash function is $H : X_1 \times X_2 \rightarrow Y$.*

$$\left| \Pr[\text{REPROG}_1^{\mathcal{D}} = 1] - \Pr[\text{REPROG}_0^{\mathcal{D}} = 1] \right| \leq \frac{3R}{2} \sqrt{\frac{q}{|X_1|}},$$

REPROG_b

- (1) $O_0 \leftarrow_{\$} Y^{X_1 \times X_2}$.
 - (2) $O_1 := O_0$.
 - (3) $b' \leftarrow \mathcal{A}^{(O_b)}, \text{Repro}$.
 - (4) Return b' .
- Repro(x_2)
- (1) $(x_1, y) \leftarrow_{\$} X_1 \times Y$.
 - (2) $O_1 := O_1^{(x_1 || x_2) \rightarrow y}$.
 - (3) Return x_1 .

Figure 1: Experiment of reprogramming.

in which the REPROG_b game for $b = 0/1$ is defined by Fig 1.

Although the Lemma 2.7 obtains a better bound in reprogramming, it is impossible to simulate the adversary in two stages. It will be unavoidable to get a q^2 loss when calling for a more general case. Hence, we use the following Lemma from [10] to complete the reduction from CCB to ν -SelfTargetMSIS.

LEMMA 2.8 (THEOREM 2 IN [10]). *Let X and Y be two finite sets, and adversary $\mathcal{A}$ be an arbitrary oracle quantum algorithm making at most Q queries to the random oracle $H : X \rightarrow Y$. The adversary outputs some $x \in X$ and a (possibly quantum) output z . The two-stage algorithm $\mathcal{S}^{\mathcal{A}}$ outputs $x \in X$ in the first stage, takes $\Theta \in Y$ as input to the second stage, and subsequently outputs a (possibly quantum) z , so that for any $x_0 \in X$ and any (possibly quantum) predicate V :*

$$\begin{aligned} \Pr[x = x_0 \wedge V(x, \Theta, z) : (x, z) \leftarrow \langle \mathcal{S}^{\mathcal{A}}, \Theta \rangle] \\ \geq \frac{1}{(2Q+1)^2} \Pr[x = x_0 \wedge V(x, H(x), z) : (x, z) \leftarrow \mathcal{A}^H]. \end{aligned}$$

Furthermore, $\mathcal{S}$ runs in time polynomial in Q , $\log |X|$ and $\log |Y|$.

3 SelfTargetMSIS variant on multiple hash target

The reduction from a single target SelfTargetMSIS to MLWE was given in [14]. The first step is to define a “plain” version of SelfTargetMSIS, known as Plain-SelfTargetMSIS, where the input matrix is not given in Hermite Normal Form. Next, it defines two experiments: the chosen coordinate binding experiment CCB and the collapsing experiment Collapse. The chain of reduction works as:

$$\begin{aligned} \text{SelfTargetMSIS}_{H, \tau, m, l, \gamma} &\leftarrow \text{Plain-SelfTargetMSIS}_{H, \tau, m, m+l, \gamma} \\ &\leftarrow \text{CCB}_{\tau, m, m+l, \gamma} \leftarrow \text{Collapse}_{m, m+l, \gamma} \leftarrow \text{MLWE}_{m+l, m, \eta}, \end{aligned}$$

of which the first transition requires $q \equiv 1 \pmod{2n}$ and the final transition requires $q > 16$ and $2\gamma\eta n(m+l) < \lfloor q/32 \rfloor$.

The above result could not be directly used in a multi-signature scheme, because the original SelfTargetMSIS and Plain-SelfTargetMSIS only target one hash result, but multi-signature further requires each party to answer an individual challenge c_i independently. More importantly, the adversary is able to control what the public keys of the malicious parties are. So, we propose a variant of the original SelfTargetMSIS to a ν -targets ν -SelfTargetMSIS.

To prove the hardness of ν -SelfTargetMSIS, we have to follow the route of [14], and consequently propose ν -Plain-SelfTargetMSIS. We can reduce the CCB experiment to our ν -Plain-SelfTargetMSIS.

Hence, we can reuse the transition from CCB, Collapse to MLWE in [14].

Definition 3.1 (ν -SelfTargetMSIS). For ν as a positive integer, $m, k, \gamma \in \mathbb{N}$ and $H : \{0, 1\}^* \rightarrow B_\tau$, the advantage that any adversary breaking the ν -SelfTargetMSIS is defined as

$$\begin{aligned} \text{Adv}_{H, \tau, m, k, \gamma}^{\nu\text{-SelfTargetMSIS}}(\mathcal{A}) := \\ \Pr \left[\begin{array}{l} \|y\|_\infty \leq \gamma \wedge \|y'\|_\infty \leq \gamma \\ \wedge H(i, A', [I_m | A]y + A'y', M) = c_i \\ \wedge y_{m+k} = c_1 \wedge y'_i = c_{i+1} \end{array} \middle| \begin{array}{l} A \leftarrow \mathcal{R}_q^{m \times k}; \\ (y, A', y', M) \leftarrow \mathcal{A}^{(H)}(A) \end{array} \right], \end{aligned}$$

in which $i \in [\nu - 1]$, $y \in \mathcal{R}_q^{m+k}$, $A' \in \mathcal{R}_q^{m \times (\tau-1)}$ and $y' \in \mathcal{R}_q^{\nu-1}$.

Definition 3.2 (ν -Plain-SelfTargetMSIS). For ν as a positive integer, $m, k, \gamma \in \mathbb{N}$ and $H : \{0, 1\}^* \rightarrow B_\tau$, the advantage that any adversary breaking the ν -Plain-SelfTargetMSIS is defined as

$$\begin{aligned} \text{Adv}_{H, \tau, m, k, \gamma}^{\nu\text{-Plain-SelfTargetMSIS}}(\mathcal{A}) := \\ \Pr \left[\begin{array}{l} \|y\|_\infty \leq \gamma \wedge \|y'\|_\infty \leq \gamma \\ \wedge H(i, A', Ay + A'y', M) = c_i \\ \wedge y_k = c_1 \wedge y'_i = c_{i+1} \end{array} \middle| \begin{array}{l} A \leftarrow \mathcal{R}_q^{m \times k}; \\ (y, A', y', M) \leftarrow \mathcal{A}^{(H)}(A) \end{array} \right], \end{aligned}$$

in which $i \in [\nu - 1]$, $y \in \mathcal{R}_q^k$, $A' \in \mathcal{R}_q^{m \times (\tau-1)}$, and $y' \in \mathcal{R}_q^{\nu-1}$.

3.1 Reducing multi-target Plain-SelfTargetMSIS to multi-target SelfTargetMSIS

PROPOSITION 3.3. *Suppose $q \equiv 1 \pmod{2n}$ and $m, k, \gamma, \tau \in \mathbb{N}$, $H_i : \{0, 1\}^* \rightarrow B_\tau$. If there is an adversary $\mathcal{A}$ making Q queries to solve ν -SelfTargetMSIS_{H, \tau, m, k, \gamma}, there is an adversary $\mathcal{B}$ solving ν -Plain-SelfTargetMSIS_{H, \tau, m, m+k, \gamma} with Q queries with loss at least n/q^k .*

Following up with Lemma 2 and Proposition 1 in [14], the proof is trivial to get by $\mathcal{R}_q \cong \mathbb{Z}_q^n$. And we remark that the randomly sampled part of our scheme is A , while A' only eliminates the probability of having no row-echelon form, other than enlarging it. So the probability bound of distinguishing two cases is less than n/q^k .

3.2 Reducing CCB to multi-target Plain-SelfTargetMSIS

We first review the experiment CCB. Let $\tau, m, l, \gamma \in \mathbb{N}$. The advantage of a quantum algorithm $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ for winning CCB_{\tau, m, l, \gamma} is defined as the probability that the experiment CCB_{\tau, m, l, \gamma} outputs 1.

The existing reduction from CCB_{\tau, m, l, \gamma} to Plain-SelfTargetMSIS_{H, \tau, m, l, \gamma} in [14] relies on the measure-and-reprogram lemma (Lemma 2.8) to build up a two-stage simulator of an adversary. The lemma seems to be unsuitable to adopt in our reduction because it is unclear whether involving more than one $(x, H(x))$ in verification would impact the lemma.

On the other hand, [10] provides a variant supporting multiple inputs x and reprogramming the corresponding outputs Θ . However, the multiplicative loss on such reduction is too high, i.e. $1/(2Q+1)^{2n}$.

```

1  $A \xleftarrow{\$} \mathcal{R}_q^{m \times l}$ .
2  $(z, T) \leftarrow \mathcal{A}_1(A)$ , where  $z \in \mathcal{R}_q^m$  and  $T$  is auxiliary register.
3  $c_i \xleftarrow{\$} B_\tau$ 
4  $y \leftarrow \mathcal{A}_2(T, c)$  where  $y \in \mathcal{R}_q^l$ .
5 if  $Ay = z, \|y\|_\infty \leq \gamma$ , and,  $y_l = c$  then
6   | return 1
7 end
8 return 0

```

Algorithm 1: Experiment $\text{CCB}_{\tau, m, l, \gamma}$.

So finding a way with minimal loss is preferable. To begin with it, we first dive into the core of Lemma 2.8.

Revisiting Lemma 2.8. The supporting technique of this lemma is Lemma 1 in [10]. The adversary $\mathcal{A}$ in the model is described as $A_1, A_2, \dots, A_Q$ with the initial state $|\phi_0\rangle$. Subsequently, the state after Q queries turns out to be $|\phi_Q^H\rangle := \mathcal{A}^H|\phi_0\rangle := A_Q O^H \dots A_1 O^H |\phi_0\rangle$, of which $O^H : |c\rangle|x\rangle|y\rangle \mapsto |c\rangle|x\rangle|y \oplus c \cdot H(x)\rangle$. The model gives controlled queries on $c \in \{0, 1\}$. Moreover, it uses the notation of reprogramming as

$$H * \Theta x_0 : x \mapsto \begin{cases} \Theta & \text{if } x = x_0, \\ H(x) & \text{Otherwise.} \end{cases}$$

where the H is the hash function modeled as a quantum random oracle. The abbreviation is $(\mathcal{A}_{i \rightarrow Q}^{H * \Theta x})(\mathcal{A}_{0 \rightarrow i}^H)|\phi_0\rangle = (\mathcal{A}_{i \rightarrow Q}^{H * \Theta x})|\phi_i^H\rangle$, meaning that the adversary is started with $|\phi_0\rangle$ and the oracle answers the first i queries with H and reprograms on x to Θ for the remaining queries. Consequently, the underlying lemma could be expressed as Lemma 3.4.

LEMMA 3.4 (LEMMA 1 IN [10]). *Let $\mathcal{A}$ be a Q -query quantum algorithm. For any $H : X \rightarrow Y$, any $x \in X$, $\Theta \in Y$, and any projection $\Pi_{x, \Theta}$, the following equation holds when $(i, b) \in \{0, \dots, Q-1\} \times \{0, 1\} \cup \{Q, 0\}$*

$$\begin{aligned} \mathbb{E}_{i, b} \left[\left\| (|x\rangle\langle x| \otimes \Pi_{x, \Theta})(\mathcal{A}_{i+b \rightarrow Q}^{H * \Theta x})(\mathcal{A}_{i \rightarrow i+b}^H)X|\phi_i^H\rangle \right\|_2^2 \right] \\ \geq \frac{\left\| (|x\rangle\langle x| \otimes \Pi_{x, \Theta})|\phi_Q^{H * \Theta x}\rangle \right\|_2^2}{(2Q+1)^2}. \end{aligned}$$

Here, $|x\rangle\langle x|$ measures the final output register X to obtain x , and the projection $\Pi_{x, \Theta}$ is the measurement of some other register Z relying on $H(x)$. Besides, the measurement of the hash query register is written as X in this formula.

In other words, the left-hand side is the expectation of measuring the query register to get x and then deciding to reprogram $H(x)$ with Θ depending on a random choice b , and finally the output register X is measured and coincides with x .

On the right-hand side of Lemma 3.4, the measurement probability that could be treated as the probability of outputting a valid z passing the verification V , as long as Θ is random.

$$\begin{aligned} \Pr[x = x_0 \wedge V(x, H(x), z) : (x, z) \leftarrow \mathcal{A}^H] \\ = \left\| (|x_0\rangle\langle x_0| \otimes \Pi_{x_0, H(x_0)})|\phi_Q^{H * \Theta x_0}\rangle \right\|_2^2. \end{aligned}$$

Now, we will adopt the Lemma 3.4 to show the reduction from CCB to v -Plain-SelfTargetMSIS.

PROPOSITION 3.5. *Let $v, m, l, \gamma, \tau \in \mathbb{N}$. Suppose there is an adversary $\mathcal{A}$ solving v -Plain-SelfTargetMSIS $_{H, \tau, m, l, \gamma}$ with at most Q queries with advantage ϵ , there exists a two-stage quantum algorithm $\mathcal{B} = (\mathcal{B}_1, \mathcal{B}_2)$ solving and winning the experiment $\text{CCB}_{\tau, m, l, \gamma}$ with advantage at least $\epsilon/(2Q+1)^2$.*

PROOF. To complete the reduction, we expect a 2-stage simulator $\mathcal{S}_1^{\mathcal{A}}$ and $\mathcal{S}_2^{\mathcal{A}}$, such that $(A', z := Ay + A'y', M, T) \leftarrow \mathcal{S}_1^{\mathcal{A}}$ and $(y, y') \leftarrow \mathcal{S}_2^{\mathcal{A}}(T, c_1)$ such that $\|y\|_\infty \leq \gamma \wedge \|y'\|_\infty \leq \gamma \wedge y_k = c_1 \wedge y'_i = c_{i+1}$.

Here, we only need to program $H(x_1)$ to $\Theta := c_1$ and remain $\{H(x_i)\}_{i>1}$ the same, which means neither H nor $H * \Theta x$ will change the result of $\{H(x_i)\}_{i>1}$. All we need to care about is whether $(1, A', Ay + A'y', M)$ has been queried or not. Moreover, when we measure the output register X for $x_1 := (1, A', Ay + A'y', M)$, we immediately determine $x_i := (i, A', Ay + A'y', M)$ for $i > 2$.

So we can use almost the same strategy as the CCB to Plain-SelfTargetMSIS reduction, that we pick a random $(i, b) \in \{0, \dots, Q-1\} \times \{0, 1\} \cup \{Q, 0\}$ and measure the query register after the i -th query, and we then choose to program the hash on or after the current query, of the measured result. And the expectation of the outcome probability that the output register X measures the same as the query register, is indeed

$$\mathbb{E}_{i, b} \left[\left\| (|x\rangle\langle x| \otimes \Pi_{x, \Theta})(\mathcal{A}_{i+b \rightarrow Q}^{H * \Theta x})(\mathcal{A}_{i \rightarrow i+b}^H)X|\phi_i^H\rangle \right\|_2^2 \right].$$

Consequently, there is a pair of simulators $(\mathcal{S}_1^{\mathcal{A}}, \mathcal{S}_2^{\mathcal{A}})$, of which $\mathcal{S}_2^{\mathcal{A}}$ receives a $\Theta := c_1$ as an input, and then outputs a valid solution to v -Plain-SelfTargetMSIS. The expectation of the probability is $\frac{1}{(2Q+1)^2}$ times the probability of outputting a single transcript, as in Lemma 3.4.

Moreover, the algorithms $\mathcal{B} = (\mathcal{B}_1, \mathcal{B}_2)$ could be constructed by $(\mathcal{S}_1^{\mathcal{A}}, \mathcal{S}_2^{\mathcal{A}})$ with the following modification.

- $\mathcal{B}_1$: calls $\mathcal{S}_1^{\mathcal{A}}$ to get (A', z, M, T) and outputs $T' := T$ and $z' := z - A'y'$. Here $y'_i = c_{i+1} = H(i, A', z, M)$ are all fixed.
- $\mathcal{B}_2$: on receiving c_1 , calls $\mathcal{S}_2^{\mathcal{A}}(T', c)$ to get (y, y') which fulfills $z = Ay + A'y' \wedge \|y\|_\infty \leq \gamma \wedge \|y'\|_\infty \leq \gamma \wedge y_k = c_1 \wedge y'_i = c_{i+1}$. Thus, it is able to output y such that $Ay = z'$.

□

3.3 Security of multi-target SelfTargetMSIS

By putting Proposition 3.3 in our paper, with Proposition 3 in [14] (reduction from the experiment Collapse to CCB) and Proposition 4 in [14] (reduction from the experiment CCB to MLWE) together, we get the following theorem on the security of v -SelfTargetMSIS.

THEOREM 3.6. *Let $m, k, \tau, \gamma, \eta \in \mathbb{N}$. Suppose $q > 16$, $q \equiv 1 \pmod{2n}$, and $2\gamma\eta n(m+k) < \lfloor q/32 \rfloor$. If there is an adversary making Q quantum queries to solve the problem v -SelfTargetMSIS $_{H, \tau, m, k, \gamma}$ with advantage ϵ and $H : \{0, 1\}^* \rightarrow B_\tau$, there exists a quantum algorithm that solves $\text{MLWE}_{m+k, m, \eta}$ with advantage*

$$\frac{\epsilon - nq^{-k}}{4(2Q+1)^2} \left(\frac{\epsilon - nq^{-k}}{4(2Q+1)^2} - \frac{1}{|B_\tau|} \right) - \frac{1}{4 \cdot 3^w},$$

Parameter	Explanation	Limitation
v	The number of signers	
n	Ring dimension	
q	Ring Modulus	$q \equiv 1 \pmod{2n}, q > 16$
(k, l)	Dimensions of $\mathbf{A}$	$l \leq k$
γ	Max signature coefficient	$\beta \ll \gamma$
η	Max coefficients of $\mathbf{s}^{(1)}, \mathbf{s}^{(2)}$	
β	The bound of $\ \mathbf{cs}\ _\infty, \mathbf{s} \in S_\eta$	$\beta = \tau\eta$
τ	Challenge space	

Table 1: Parameters for QROM-MuSig

for all $w \in \mathbb{N}$.

4 QROM-MuSig

In this section, we propose the QROM-MuSig, which is highly efficient to run NTT because of $q \equiv 1 \pmod{2n}$. All the parameter requirements follow Table 1. Moreover, we consider the challenge space as $\text{ChSet} = B_\tau$, and thus the $|\text{ChSet}| = 2^\tau \binom{n}{\tau}$.

4.1 Construction

Setup. On input a security parameter 1^λ , it outputs the public parameters as shown in Table 1. This part also requires the generation of the public matrix $\mathbf{A}$. A safe generation requires all participants to efficiently check the trustworthiness of $\mathbf{A}$. Here we refer to [1] for generating a common reference string without trusted setup. It also defines two hash functions $H_1 : \{0, 1\}^* \rightarrow \{0, 1\}^*$ and $H_2 : \{0, 1\}^* \rightarrow B_\tau$.

Key generation. On input the public parameter, it samples secret keys by the MLWE parameters $(\mathbf{s}^{(1)}, \mathbf{s}^{(2)}) \leftarrow S_\eta^\ell \times S_\eta^k$. It then constructs the public key $\mathbf{t} := \mathbf{A}\mathbf{s}^{(1)} + \mathbf{s}^{(2)}$. It will finally output $\text{pk} := (\mathbf{t}, \text{sk} := (\mathbf{s}^{(1)}, \mathbf{s}^{(2)}))$.

Signing. Assuming that the multi-signature is targeting the message μ , the size of signers is v , the member index is ranged from 1 to v , and the current party is indexed by v . Each party takes part in the signing procedure with the secret key of the current party sk_v and the collection of the public keys pk .

Sign₁. The signer generates its own random masks by sampling $\mathbf{y}_v^{(1)} \leftarrow S_{\gamma-1}^\ell$ and $\mathbf{y}_v^{(2)} \leftarrow S_{\gamma-1}^k$ first, computes $\mathbf{w}_v = \mathbf{A}\mathbf{y}_v^{(1)} + \mathbf{y}_v^{(2)}$, and makes a commitment. $\mathbf{g}_v = H_1(\text{pk}_v, \mathbf{w}_v)$. It sends out $(\text{state}_1 := (\text{pk}, \mathbf{y}_v^{(1)}, \mathbf{y}_v^{(2)}, \mathbf{w}_v), \mathbf{g}_v)$.

Sign₂. On receiving the previous state state_1 and all the commitments denoted as $\{\mathbf{g}_u\}_{u \in [v], u \neq v}$ from other parties, it outputs a reveal of the commitment as $\mathbf{w}_v$ together with the current state, i.e. $(\text{state}_2 := (\text{pk}, \mathbf{y}_v^{(1)}, \mathbf{y}_v^{(2)}, \{\mathbf{g}_u\}_{u \in [v]}, \mathbf{w}_v))$.

Sign₃. It takes the previous state state_2 and all the revealing $\{\mathbf{w}_u\}_{u \in [v]}$ as inputs. And it performs and outputs the corresponding state $(\text{state}_3 := (\text{pk}, \mathbf{w}), (\mathbf{z}_v^{(1)}, \mathbf{z}_v^{(2)}))$ with the following.

- (1) Abort if $\mathbf{g}_u \neq H_1(\text{pk}_u, \mathbf{w}_u)$ for some $u \in [v]$.
- (2) Compute $\mathbf{w} = \sum_{u=1}^v \mathbf{w}_u$.
- (3) Get a challenge $c_v = H_2(v, \mathbf{w}, \mu, \text{pk})$.
- (4) Compute $\mathbf{z}_v^{(1)} = \mathbf{y}_v^{(1)} + c_v \mathbf{s}_v^{(1)}$ and $\mathbf{z}_v^{(2)} = \mathbf{y}_v^{(2)} + c_v \mathbf{s}_v^{(2)}$.

- (5) Abort when $\|\mathbf{z}_v^{(1)}\|_\infty \geq \gamma - \beta$ or $\|\mathbf{z}_v^{(2)}\|_\infty \geq \gamma - \beta$.

Sign₄. On receiving the previous state state_3 and a series of $\{\mathbf{z}_u\}_{u \in [v]}$ from each individual signer, it sums up all of the $\mathbf{z}_u$ together by $\mathbf{z}^{(1)} = \sum_{u=1}^v \mathbf{z}_u^{(1)}$ and $\mathbf{z}^{(2)} = \sum_{u=1}^v \mathbf{z}_u^{(2)}$. It outputs the signature $\sigma = (\mathbf{w}, \mathbf{z}^{(1)}, \mathbf{z}^{(2)})$.

Verification. The verification takes the public keys of all the parties $\text{pk} = \{\text{pk}_u\}_{u \in [v]}$, and signatures in the form $\sigma = (\mathbf{w}, \mathbf{z}^{(1)}, \mathbf{z}^{(2)})$. It reconstructs the challenges $c_u = H_2(u, \mathbf{w}, \mu, \text{pk})$. And it later outputs 1 if the following two conditions are fulfilled, and it outputs 0 otherwise.

- $\|\mathbf{z}^{(1)}\|_\infty < v(\gamma - \beta)$,
- $\|\mathbf{z}^{(2)}\|_\infty < v(\gamma - \beta)$,
- $\mathbf{A}\mathbf{z}^{(1)} + \mathbf{z}^{(2)} = \mathbf{w} + \sum_{v=1}^v c_v \mathbf{t}_v$.

4.2 Security

In the security reduction, we assume that the forger has the quantum query to hash functions H_1 and H_2 , while it can only access the signing oracles classically just like the real-world classical communication channel. The main proof follows [12]. However, we remark that the original proof is relying on the lossy-key generation where $q \equiv 5 \pmod{8}$, but our scheme relies on $q \equiv 1 \pmod{2n}$.

Without loss of generality, we assume the adversary is going to interactively query with the first party that acted by the simulator, which would involve the pk_1 without knowing sk_1 . And all other parties are controlled by the adversary.

LEMMA 4.1. *If the multi-signature scheme QROM-MuSig is secure in MS-UF-NMA model, the QROM-MuSig is also secure in MS-UF-CMA with the advantage loss $\frac{3Q_s E}{2} \sqrt{\frac{Q_H + Q_s v E}{2^\tau \binom{n}{\tau}}}$, of which Q_s is the number of queries on signing, Q_H is the number of queries on hash, E is the expectation of the repeating time in signing.*

PROOF. The major technique of achieving unforgeability is in game-hopping. Here is the definition of the original game.

Setup. On input 1^λ , the simulator sets up $\mathbf{A}$, and the key pair for the first party, i.e. $(\text{pk}^*, \text{sk}^*) := (\text{pk}_1, \text{sk}_1) \leftarrow \text{KeyGen}(\mathbf{A})$. All public parameters and pk^* are given to the adversary $\mathcal{A}$. We write the secret key of the first party as $(\mathbf{s}_1^*, \mathbf{s}_2^*) = \text{sk}^*$.

Oracles. Hash functions H_1, H_2 are simulated by quantum random oracles. The adversary is able to make Q_1 queries to H_1 and Q_H queries to H_2 .

SigningOracle. Suppose the signing query is denoted by (pk, μ) . If pk^* is not contained by pk , it just answers the query normally. Otherwise, the index of the target keys pk^* is set to the first one without loss of generality (i.e., $\text{pk}_1 = \text{pk}^*$). Moreover, we assume that each state of the signing procedure is continuous and each state_i would be unique as if the adversary is interacting with a normal user.

- **Sign₁.** The simulator invokes the original Sign_1 and sends the commitment $\mathbf{g}_1$ to $\mathcal{A}$ as Party 1.

- Sign_2 . The simulator receives $\{g_u\}_{u \in [2, v]}$ from $\mathcal{A}$ and works as the party 1 to invoke the original Sign_2 and sends the committed message w_1 to $\mathcal{A}$.
- Sign_3 . The simulator receives $\{w_u\}_{u \in [2, v]}$ from $\mathcal{A}$, and invokes the original Sign_3 and sends $z_u^{(1)}$ and $z_u^{(2)}$ to $\mathcal{A}$.
- Sign_4 . The simulator receives $\{z_u^{(1)}, z_u^{(2)}\}_{u \in [2, v]}$ from $\mathcal{A}$, and works as the party 1 to invoke the original Sign_4 and sends $(w, z^{(1)}, z^{(2)})$ to $\mathcal{A}$.

Challenge. The forger generates a valid signature of any message on the tuple $(pk^*, (w^*, z_1^*, z_2^*), \mu^*)$ such that it passes the verification $\text{Verify}(pk^*, (w^*, z_1^*, z_2^*), \mu^*) = 1$ and $pk^* \in \mathcal{PK}^*$.

The target of the game hopping is to replace the keys of the first party by the underlying MS-UF-NMA key and make the simulator able to answer any query even without the secret key.

Game₀: The original game.

Game₁: It is the same as Game₀, except:

- **Oracles.** The hash function H_1 is replaced by a $2(Q_1 + Q_s vE)$ degree polynomial function $\mathcal{H}_1$, of which Q_1 is the number of queries that H_1 is asked directly and Q_s is the number of queries that signing oracle is invoked.

By Lemma 2.1, we have:

$$|\Pr[\mathcal{A}^{\text{Game}_0} = 1] - \Pr[\mathcal{A}^{\text{Game}_1} = 1]| \leq \text{negl}(\lambda).$$

Game₂: It is the same as Game₁, except:

- Sign_2 . The simulator receives $\{g_u\}_{u \in [2, v]}$ from $\mathcal{A}$. To generate w_1 , the simulator extracts the original w_u for $u \in [2, v]$ by the computing the roots of $\mathcal{H}_1 - g_u$ as r_u . The probability that the number of roots greater than 2 is given by ϵ_{roots} . If there is more than one result, it just simply picks a random one. And it sends w_1 to $\mathcal{A}$.
- Sign_3 . The simulator receives $\{w_u\}_{u \in [2, v]}$ from $\mathcal{A}$, and checks the extraction is indeed the original (pk_u, w_u) . It aborts if the extraction is not correct. Subsequently, it invokes the original Sign_3 and sends $z_1^{(1)}$ and $z_1^{(2)}$ to $\mathcal{A}$.

We append the public key into the hash input such that it would avoid multiple roots with high probability. The extraction loss of taking the polynomial roots is very small by using polynomial simulation and Lemma 2.1. Hence we have:

$$|\Pr[\mathcal{A}^{\text{Game}_1} = 1] - \Pr[\mathcal{A}^{\text{Game}_2} = 1]| \leq \text{negl}(\lambda).$$

Game_{3,i} for $i = 0, \dots, Q_s E$: We consider the series of sub-hoppings as each individual signing query without restarting. When the restarts occur, the simulator just simply jumps to the next sub-game and thus the expected number of sub-games turns to the product of the query number and the expected restart number. Before any hopping under the sub-game, the original sub-game Game_{3,0} is indeed the same as the previous game Game₂.

Following up with the previous state, Game_{3,i} transits from Game_{3,i-1}. In this hopping, the simulator will pick a random challenge c_1 in the very beginning, which the simulator is able to answer the following up message with w . And using Lemma 2.7, the hopping would succeed with non-negligible probability.

Here are the changes from Game_{3,i-1} to Game_{3,i}. The i -th signing call is simulated by the following.

- Sign_1 . The simulator picks a random challenge c_1 for the first party and $z_1 \leftarrow S_{\gamma-\beta-1}^\ell$ and $z_2 \leftarrow S_{\gamma-\beta-1}^k$. Subsequently, it sets $w_1 = Az_1^{(1)} + z_1^{(2)} - c_1 t_1$ with probability $1 - e^{-n\beta/\gamma}$ and $w_1 \leftarrow \mathcal{R}_q^k$. And it computes and sends g_1 as the original algorithm.
- Sign_2 . As $\{w_u\}_{u \in [2, v]}$ are successfully extracted, the simulator construct w . Then the simulator will reprogram the hash $H_2(1, w, \mu, pk)$ as c_1 .
- Sign_3 . If w_1 is sampled uniformly, the simulator restarts the algorithm. Alternatively, it will proceed the original Sign_3 .

If $\|c_i s_i^{(1)}\|_\infty \leq \beta$, this scheme is zero-knowledge about the secret key. All the information that the signers produce is $(w_i, z_i^{(1)}, z_i^{(2)})$. The probability $z_i^{(1)}$ together with c_i producing is

$$\Pr[z_i^{(1)}, c_i] = \Pr[c_i] \cdot \Pr[y_i^{(1)} = z_i^{(1)} - c_i s_i^{(1)} | c_i]$$

Because of the bound $\|c_i s_i^{(1)}\|_\infty \leq \beta$ and the bound $\|z_i^{(1)}\|_\infty < \gamma - \beta$, we have $\|z_i^{(1)} - c_i s_i^{(1)}\|_\infty \leq \gamma - 1$ and all these values are valid for a $y_i^{(1)}$. As a result, for any singular $z_i^{(1)}$ with $\|z_i^{(1)}\|_\infty < \gamma - \beta$, the probability distribution of $(z_i^{(1)}, c_i)$ is uniformly in $S_{\gamma-\beta-1}^\ell \times \mathcal{B}_\tau$. And this statement also holds for $(y_i^{(2)}, c_i)$ in $S_{\gamma-1}^k \times \mathcal{B}_\tau$ because it is generated independently of $y_i^{(1)}$. Thus, the distribution of $(z_i^{(1)}, z_i^{(2)}, c_i)$ distributes uniformly in $S_{\gamma-\beta-1}^\ell \times S_{\gamma-\beta-1}^k \times \mathcal{B}_\tau$. Moreover, w_i is identical to the original signing by

$$w_i = Ay_i^{(1)} + y_i^{(2)} = A(z_i^{(1)} - c_i s_i^{(1)}) + (z_i^{(2)} - c_i s_i^{(2)}) = Az_i^{(1)} + z_i^{(2)} - c_i t_i.$$

In each individual sub-game hopping, there is only one point under H_2 to be reprogrammed, and the total number of hash queries is $Q_H + Q_s vE$. The random challenge space is $\mathcal{B}_\tau$ of size $2^\tau \binom{n}{\tau}$. As a result, we have

$$|\Pr[\mathcal{A}^{\text{Game}_{3,i}} = 1] - \Pr[\mathcal{A}^{\text{Game}_{3,i-1}} = 1]| \leq \frac{3}{2} \sqrt{\frac{Q_H + Q_s vE}{2^\tau \binom{n}{\tau}}}$$

Summing up all sub-games together, the difference turns to

$$\begin{aligned} & |\Pr[\mathcal{A}^{\text{Game}_3} = 1] - \Pr[\mathcal{A}^{\text{Game}_2} = 1]| \\ &= \sum_{i=1}^{Q_s E} |\Pr[\mathcal{A}^{\text{Game}_{3,i}} = 1] - \Pr[\mathcal{A}^{\text{Game}_{3,i-1}} = 1]| \\ &\leq \frac{3Q_s E}{2} \sqrt{\frac{Q_H + Q_s vE}{2^\tau \binom{n}{\tau}}}. \end{aligned}$$

Game₄: At the final game, we notice that the signing oracle queries could be perfectly simulated without using any private information of the first party. Now consider an algorithm $\mathcal{B}$ playing the MS-UF-NMA game and receiving (pp, pk^*) . Then $\mathcal{B}$ can run Game₄ with the MS-UF-CMA adversary $\mathcal{A}$ using (pp, pk^*) . If $\mathcal{A}$ wins by outputting (PK^*, σ^*, m^*) , then $\mathcal{B}$ also wins by outputting

$(\mathbf{PK}^*, \sigma^*, m^*)$. Hence we have:

$$\begin{aligned} & \text{Adv}_{\text{QROM-Musig}}^{\text{MS-UF-CMA}}(\mathcal{A}) \\ & \leq \text{Adv}_{\text{QROM-Musig}}^{\text{MS-UF-NMA}}(\mathcal{B}) + \frac{3Q_s E}{2} \sqrt{\frac{Q_H + Q_s vE}{2^{\tau} \binom{n}{\tau}}} + \text{negl}(\lambda). \end{aligned}$$

□

Finally, we show that our multi-signature scheme is MS-UF-NMA secure.

LEMMA 4.2. *Let $\zeta = v(\gamma_1 - \beta - 1)$. If there is any adversary who breaks MS-UF-NMA of QROM-Musig, there exists an adversary that breaks $\text{MLWE}_{k,\ell,\eta}$ or $v\text{-SelfTargetMSIS}_{H,\tau,k,\ell+1,\zeta}$.*

PROOF. From the point of verification, if there is any attack giving out $(\mathbf{pk}, \mathbf{w}, \mathbf{z}^{(1)}, \mathbf{z}^{(2)})$. The following equation must be correct for

$$\begin{aligned} & \bullet \|\mathbf{z}^{(1)}\|_{\infty} < v(\gamma - \beta) \text{ and } \|\mathbf{z}^{(2)}\|_{\infty} < v(\gamma - \beta), \\ & \bullet \mathbf{Az}^{(1)} + \mathbf{z}^{(2)} = \mathbf{w} + \sum_{i=1}^v c_i \mathbf{t}_i \end{aligned}$$

, in which the challenges are computed from the following formula

$$c_u = H_2(u, \mathbf{Az}^{(1)} + \mathbf{z}^{(2)} - \sum_{i=1}^v c_i \mathbf{t}_i, \mu, \mathbf{pk}).$$

It is equivalent to find out

$$\mathbf{y}_1 = [\mathbf{z}^{(2)} \quad \mathbf{z}^{(1)} \quad \mathbf{c}_1]^\top, \mathbf{y}_2 = [\mathbf{c}_2 \quad \cdots \quad \mathbf{c}_v]^\top,$$

and there is a

$$\mathbf{A}'_1 = [\mathbf{I} \quad \mathbf{A} \quad \mathbf{t}_1], \mathbf{A}'_2 = [\mathbf{t}_2 \quad \cdots \quad \mathbf{t}_v],$$

such that $H_2(u, \mathbf{A}'_1 \mathbf{y}_1 + \mathbf{A}'_2 \mathbf{y}_2, \mu, \mathbf{pk}) = c_u$.

If the adversary could distinguish $\mathbf{A}'_1$ from the uniformly generated $[\mathbf{I} \quad \mathbf{A}]$, it would break $\text{MLWE}_{k,\ell,\eta}$. Otherwise, it turns out to be indeed the same form of $\tau\text{-SelfTargetMSIS}_{H,\tau,k,\ell+1,\zeta}$, except that $\mathbf{pk}$ contains more information than $\mathbf{A}'$ in $\tau\text{-SelfTargetMSIS}$. This change will not affect the security of such a problem, because $\mathbf{A}$ is fixed before the $\mathcal{A}$ could make an attack. Thus,

$$\begin{aligned} & \text{Adv}_{\text{QROM-Musig}}^{\text{MS-UF-NMA}}(\mathcal{A}) \\ & \leq \text{Adv}_{k,\ell,\eta}^{\text{MLWE}}(\mathcal{A}_1) + \text{Adv}_{H,\tau,k,\ell+1,\zeta}^{v\text{-SelfTargetMSIS}}(\mathcal{A}_2) + \text{negl}(\lambda). \end{aligned}$$

□

4.2.1 Comparison. We remark that our proof strategy has a better loss compared to the existing scheme [12]. Their scheme uses [22] to reprogram and further answers the signing queries, while our scheme uses Lemma 2.7. Because the simulation of [22] requires a $2Q$ -wise independent function, and the loss is $(4 + \sqrt{2})\sqrt{Q}2^{-\theta/4}$. Here θ is the input collision-entropy. And to further control the collision-entropy, they require an almost-compressed random oracle to simulate the hash input which works like $c_i := H_1(H_2(\cdot))$. By adding up with the almost compressed oracle failure probability, their loss is $\frac{2\sqrt{Q_1}}{2^{\theta/2}} + \frac{Q_s E(4 + \sqrt{2})\sqrt{(Q_2 + Q_s E - 1)v}}{2^{\theta/4}}$ for the reprogramming part, while our scheme gets $\frac{3Q_s E}{2} \sqrt{\frac{Q_H + Q_s vE}{2^{\tau} \binom{n}{\tau}}}$ instead. Here θ could be simply treated as the input length of H_2 .

Although it seems to be controllable that the loss decreases when the θ increases, the length-preserving property of [22] prevents it

from choosing the θ freely. Moreover, we would expect the challenge space to be B_τ when using Dilithium, which makes it hard to choose a proper θ in reality. Instead, using Lemma 2.7 does not limit the relationship between the input space and the output space and could guide the parameter choosing in a better way.

5 Conclusion

In this paper, we introduce the $v\text{-SelfTargetMSIS}$ problem and present a reduction from CCB to it. Consequently, we could further show that $v\text{-SelfTargetMSIS}$ is at least as hard as the MLWE problem. Besides, we propose a multi-signature scheme based on the one proposed by [12]. Our multi-signature scheme can reach a much tighter loss by using $v\text{-SelfTargetMSIS}$ and [13], and it is proven in $q \equiv 1 \pmod{2n}$ that it benefits from NTT acceleration. Thus, our scheme is more practical and is expected to be widely used.

References

- [1] Damiano Abram, Brent Waters, and Mark Zhandry. 2023. Security-Preserving Distributed Samplers: How to Generate Any CRS in One Round Without Random Oracles. In *CRYPTO 2023 (Lecture Notes in Computer Science, Vol. 14081)*, Helena Handschuh and Anna Lysyanskaya (Eds.). Springer, 489–514. https://doi.org/10.1007/978-3-031-38557-5_16
- [2] Andris Ambainis, Mike Hamburg, and Dominique Unruh. 2019. Quantum Security Proofs Using Semi-classical Oracles. In *CRYPTO 2019 (Lecture Notes in Computer Science, Vol. 11693)*, Alexandra Boldyreva and Daniele Micciancio (Eds.). Springer, 269–295. https://doi.org/10.1007/978-3-030-26951-7_10
- [3] Shi Bai, L  o Ducas, Eike Kiltz, Tancre  de Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehl  . 2020. CRYSTALS-Dilithium: Algorithm Specification and Supporting Documentation (Version 3.1). Round-3: Submission to the NIST Post-Quantum Cryptography Standardization Project. <https://cryptojedi.org/papers/dilithiumnistr3>.
- [4] Dan Boneh,   zg  r Dagdelen, Marc Fischlin, Anja Lehmann, Christian Schaffner, and Mark Zhandry. 2011. Random Oracles in a Quantum World. In *ASIACRYPT 2011 (Lecture Notes in Computer Science, Vol. 7073)*, Dong Hoon Lee and Xiaoyun Wang (Eds.). Springer, 41–69. https://doi.org/10.1007/978-3-642-25385-0_3
- [5] Cecilia Boschini, Akira Takahashi, and Mehdi Tibouchi. 2022. MuSig-L: Lattice-Based Multi-signature with Single-Round Online Phase. In *CRYPTO 2022 (Lecture Notes in Computer Science, Vol. 13508)*, Yevgeniy Dodis and Thomas Shrimpton (Eds.). Springer, 276–305. https://doi.org/10.1007/978-3-031-15979-4_10
- [6] Katharina Boudgoust and Akira Takahashi. 2023. Sequential Half-Aggregation of Lattice-Based Signatures. In *Computer Security - ESORICS 2023 - 28th European Symposium on Research in Computer Security, The Hague, The Netherlands, September 25-29, 2023, Proceedings, Part I (Lecture Notes in Computer Science, Vol. 14344)*, Gene Tsudik, Mauro Conti, Kaitai Liang, and Georgios Smaragdakis (Eds.). Springer, 270–289. https://doi.org/10.1007/978-3-031-50594-2_14
- [7] Yanbo Chen. 2023. DualMS: Efficient Lattice-Based Two-Round Multi-signature with Trapdoor-Free Simulation. In *CRYPTO 2023 (Lecture Notes in Computer Science, Vol. 14085)*, Helena Handschuh and Anna Lysyanskaya (Eds.). Springer, 716–747. https://doi.org/10.1007/978-3-031-38554-4_23
- [8] Kai-Min Chung, Serge Fehr, Yu-Hsuan Huang, and Tai-Ning Liao. 2021. On the Compressed-Oracle Technique, and Post-Quantum Security of Proofs of Sequential Work. In *EUROCRYPT 2021 (Lecture Notes in Computer Science, Vol. 12697)*, Anne Canteaut and Fran  ois-Xavier Standaert (Eds.). Springer, 598–629. https://doi.org/10.1007/978-3-030-77886-6_21
- [9] Ivan Damg  rd, Claudio Orlandi, Akira Takahashi, and Mehdi Tibouchi. 2021. Two-Round n-out-of-n and Multi-signatures and Trapdoor Commitment from Lattices. In *PKC 2021 (Lecture Notes in Computer Science, Vol. 12710)*, Juan A. Garay (Ed.). Springer, 99–130. https://doi.org/10.1007/978-3-030-75245-3_5
- [10] Jelle Don, Serge Fehr, and Christian Majenz. 2020. The Measure-and-Reprogram Technique 2.0: Multi-round Fiat-Shamir and More. In *CRYPTO 2020 (Lecture Notes in Computer Science, Vol. 12172)*, Daniele Micciancio and Thomas Ristenpart (Eds.). Springer, 602–631. https://doi.org/10.1007/978-3-030-56877-1_21
- [11] L  o Ducas, Eike Kiltz, Tancre  de Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehl  . 2018. CRYSTALS-Dilithium: A Lattice-Based Digital Signature Scheme. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2018, 1 (2018), 238–268. <https://doi.org/10.13154/TCHES.V2018.I1.238-268>
- [12] Masayuki Fukumitsu and Shingo Hasegawa. 2020. A Lattice-Based Provably Secure Multisignature Scheme in Quantum Random Oracle Model. In *ProvSec 2020 (Lecture Notes in Computer Science, Vol. 12505)*, Khoa Nguyen, Wenling Wu, Kwok-Yan Lam, and Huaxiong Wang (Eds.). Springer, 45–64. <https://doi.org/10.1007/978->

- 3-030-62576-4_3
- [13] Alex B. Grilo, Kathrin Hövelmanns, Andreas Hülsing, and Christian Majenz. 2021. Tight Adaptive Reprogramming in the QROM. In *ASIACRYPT 2021 (Lecture Notes in Computer Science, Vol. 13090)*, Mehdi Tibouchi and Huaxiong Wang (Eds.). Springer, 637–667. https://doi.org/10.1007/978-3-030-92062-3_22
 - [14] Kelsey A. Jackson, Carl A. Miller, and Daochen Wang. 2024. Evaluating the Security of CRYSTALS-Dilithium in the Quantum Random Oracle Model. In *EUROCRYPT 2024 (Lecture Notes in Computer Science, Vol. 14656)*, Marc Joye and Gregor Leander (Eds.). Springer, 418–446. https://doi.org/10.1007/978-3-031-58751-1_15
 - [15] Eike Kiltz, Vadim Lyubashevsky, and Christian Schaffner. 2018. A Concrete Treatment of Fiat-Shamir Signatures in the Quantum Random-Oracle Model. In *EUROCRYPT 2018 (Lecture Notes in Computer Science, Vol. 10822)*, Jesper Buus Nielsen and Vincent Rijmen (Eds.). Springer, 552–586. https://doi.org/10.1007/978-3-319-78372-7_18
 - [16] Qiqi Lai, Feng-Hao Liu, Yang Lu, Haiyang Xue, and Yong Yu. 2024. Scalable Two-Round n -out-of- n and Multi-Signatures from Lattices in the Quantum Random Oracle Model. Cryptology ePrint Archive, Paper 2024/1574. <https://eprint.iacr.org/2024/1574>
 - [17] Qipeng Liu and Mark Zhandry. 2019. Revisiting Post-quantum Fiat-Shamir. In *CRYPTO 2019 (Lecture Notes in Computer Science, Vol. 11693)*, Alexandra Boldyreva and Daniele Micciancio (Eds.). Springer, 326–355. https://doi.org/10.1007/978-3-030-26951-7_12
 - [18] Vadim Lyubashevsky. 2009. Fiat-Shamir with Aborts: Applications to Lattice and Factoring-Based Signatures. In *ASIACRYPT 2009 (Lecture Notes in Computer Science, Vol. 5912)*, Mitsuru Matsui (Ed.). Springer, 598–616. https://doi.org/10.1007/978-3-642-10366-7_5
 - [19] Vadim Lyubashevsky and Gregory Neven. 2017. One-Shot Verifiable Encryption from Lattices. In *EUROCRYPT 2017 (Lecture Notes in Computer Science, Vol. 10210)*, Jean-Sébastien Coron and Jesper Buus Nielsen (Eds.). 293–323. https://doi.org/10.1007/978-3-319-56620-7_11
 - [20] Vadim Lyubashevsky and Gregor Seiler. 2018. Short, Invertible Elements in Partially Splitting Cyclotomic Rings and Applications to Lattice-Based Zero-Knowledge Proofs. In *EUROCRYPT 2018 (Lecture Notes in Computer Science, Vol. 10820)*, Jesper Buus Nielsen and Vincent Rijmen (Eds.). Springer, 204–224. https://doi.org/10.1007/978-3-319-78381-9_8
 - [21] National Institute of Standards and Technology. 2024. *Module-Lattice-Based Digital Signature Standard*. Technical Report Federal Information Processing Standards Publications (FIPS) 204. U.S. Department of Commerce, Washington, D.C.
 - [22] Dominique Unruh. 2015. Non-Interactive Zero-Knowledge Proofs in the Quantum Random Oracle Model. In *EUROCRYPT 2015 (Lecture Notes in Computer Science, Vol. 9057)*, Elisabeth Oswald and Marc Fischlin (Eds.). Springer, 755–784. https://doi.org/10.1007/978-3-662-46803-6_25
 - [23] John Watrous. 2009. Zero-Knowledge against Quantum Attacks. *SIAM J. Comput.* 39, 1 (2009), 25–58. <https://doi.org/10.1137/060670997>
 - [24] Mark Zhandry. 2012. Secure Identity-Based Encryption in the Quantum Random Oracle Model. In *CRYPTO 2012 (Lecture Notes in Computer Science, Vol. 7417)*, Reihaneh Safavi-Naini and Ran Canetti (Eds.). Springer, 758–775. https://doi.org/10.1007/978-3-642-32009-5_44
 - [25] Mark Zhandry. 2019. How to Record Quantum Queries, and Applications to Quantum Indifferentiability. In *CRYPTO 2019 (Lecture Notes in Computer Science, Vol. 11693)*, Alexandra Boldyreva and Daniele Micciancio (Eds.). Springer, 239–268. https://doi.org/10.1007/978-3-030-26951-7_9